PetVerse

Informe técnico

Red social para mascotas con Flutter Web y Firebase

Arquitectura · Modelo de datos · Seguridad
Validación · Despliegue

Proyecto Diseño e Implementación de Sitios Web Dinámicos

Docente: Prof. Jorge Fabián Siles Guzmán
Consigna: 14 de octubre de 2026
Informe: 20 de octubre de 2026
Versión de la aplicación: 0.1.0+1

Despliegue

https://petverse-29db2.web.app/

Repositorio

https://github.com/joacocop/PetVerse

1 Resumen del proyecto

PetVerse es una aplicación web progresiva desarrollada con Flutter Web y Firebase para administrar perfiles y mascotas, compartir publicaciones, gestionar avisos de adopción y mantener conversaciones privadas. La solución separa la interfaz, el estado de aplicación y el acceso a los datos para poder verificar cada responsabilidad sin depender de una única sesión de producción.

El frontend está publicado en Firebase Hosting y su acceso público por HTTPS fue comprobado el 20 de octubre de 2026. La verificación alcanzó la página de entrada y los artefactos principales del despliegue. No se realizaron nuevas operaciones con cuentas reales ni una auditoría de la configuración remota de Firebase durante la elaboración de este informe.

Objetivo y alcance

El objetivo general es implementar una red social de mascotas con persistencia, actualizaciones en tiempo real y una experiencia adaptable a distintos dispositivos. El trabajo desarrolla los catorce puntos de verificación de la consigna y documenta por separado el código existente, su validación local y la activación efectiva de los servicios externos.

El alcance acordado excluye Apple como proveedor de acceso y utiliza Firebase Hosting en lugar de Vercel. Se mantiene el proyecto sin facturación. Por esa decisión, la subida a Cloud Storage permanece desactivada y el backend de notificaciones automáticas no está desplegado. Estas restricciones afectan el cumplimiento operativo de PV8 y PV10 y no se consideran resueltas por disponer del código.

Organización del informe

2 y 3 Trazabilidad de requisitos y alcance funcional

4 Arquitectura y organización del código

5 y 6 Modelo de datos y consistencia transaccional

7 Seguridad y privacidad

8 Gamificación

9 PWA y diseño adaptable

10 Despliegue y operación

11 Estrategia de pruebas y resultados

12 Evidencia visual

13 Dificultades y riesgos pendientes

14 Conclusiones y fuentes de referencia

2 Trazabilidad de PV1 a PV7

La matriz relaciona cada requisito con el comportamiento implementado y la evidencia disponible. “Implementado” describe el código y las verificaciones indicadas; no implica que se hayan repetido todos los escenarios sobre el backend publicado.

PV	Requisito	Implementación y verificación
PV1	Autenticación	Registro y acceso con correo y contraseña, recuperación de contraseña y Google mediante Firebase Auth. Rutas privadas protegidas por la sesión. El acceso real fue confirmado durante la configuración; las pruebas automatizadas ejercitan el adaptador demo. Apple está excluido del alcance.
PV2	Perfil editable	Nombre, fotografía, biografía y lista de mascotas del titular. Se separa el perfil privado del directorio público mínimo. Las reglas impiden que terceros accedan al documento privado.
PV3	Registro de mascotas	Alta y edición de nombre, tipo, raza, edad, peso, fotografías y biografía. Las mascotas quedan bajo users/{uid}/pets. Los flujos de formularios se comprueban en pruebas de widgets.
PV4	Feed de publicaciones	Contenido con imagen o video, texto, autor, mascota asociada y fecha del servidor. Consulta ordenada por createdAt descendente y escucha mediante snapshots de Firestore. Admite medios mediante URL.
PV5	Likes en tiempo real	Reacción por botón o doble toque, animación y contador derivado de los UID que reaccionaron. Transformaciones atómicas y transacción para el crédito de puntos. Se prueban concurrencia e idempotencia.
PV6	Comentarios	Alta, lectura en tiempo real y eliminación exclusiva por su autor. Mensajes de hasta 1000 caracteres. Los emuladores verifican suplantación rechazada y recepción entre dos clientes.
PV7	Adopciones	Grilla con filtros combinados por tipo, edad y ubicación; detalle y formulario de contacto. El contacto abre mensajería privada con el responsable. El propietario puede cerrar el aviso.

Integración de los módulos

La mascota registrada es la referencia para crear publicaciones y avisos de adopción. La identidad se toma de la sesión activa; no se confía únicamente en el usuario indicado por un formulario. El contacto desde una adopción reutiliza el mismo servicio de conversaciones que el directorio de usuarios, evitando mantener dos historiales independientes.

3 Trazabilidad de PV8 a PV14

PV	Requisito	Implementación y estado operativo
PV8	Subida de archivos	Selector local, validación, progreso y adaptador Firebase Storage implementados. Reglas y subida comprobadas en emulador. Desactivado en la configuración de producción mientras se mantenga la decisión de no habilitar facturación. Una URL externa no sustituye la subida solicitada.
PV9	Mensajería privada	Conversaciones individuales con historial Firestore y escucha en vivo. Participantes inmutables y escritura atómica del mensaje y resumen. Las pruebas rechazan lectura por terceros y falsificación de remitente.
PV10	Notificaciones push	Cliente FCM, clave pública VAPID, worker y funciones para likes, comentarios y mensajes preparados. Se prueban ayudantes de generación de avisos. Las funciones no están desplegadas y no se acredita una notificación automática recibida en producción.
PV11	Gamificación	Puntos por acciones válidas, niveles e insignias derivados de créditos únicos. Reglas y pruebas impiden duplicación, edición de créditos y totales arbitrarios.
PV12	PWA instalable	Manifest, iconos, worker propio, flujo de instalación y página sin conexión. Se prueban assets y eventos del prompt. La instalación final en el sistema operativo sobre la URL pública queda como aceptación manual.
PV13	Diseño adaptable	Navegación inferior o lateral según el ancho, formularios flexibles y grillas adaptables. Pruebas de widgets aprobadas a 320, 768 y 1440 píxeles de ancho. No equivale a una auditoría de accesibilidad.
PV14	Publicación	Frontend alojado en Firebase Hosting. HTTPS y artefactos principales responden HTTP 200. La publicación de Hosting no despliega por sí sola las reglas, Storage ni Cloud Functions.

Criterio de cierre de la actividad

La versión publicada permite presentar el frontend y los módulos respaldados por Firestore y Authentication. Para acreditar íntegramente la consigna todavía debe demostrarse una subida real desde el dispositivo, al menos una notificación push automática y la instalación de la PWA. También deben completarse las capturas de los módulos restantes y el video de dos a tres minutos solicitado para la entrega.

4 Arquitectura de la solución

La aplicación adopta una arquitectura por capas con backend administrado. Flutter ejecuta la presentación en el navegador; Firebase Auth administra la identidad y Firestore persiste documentos y distribuye cambios. El acceso directo del cliente a Firebase hace que las reglas de seguridad sean parte del diseño del backend, no una validación opcional de la interfaz.

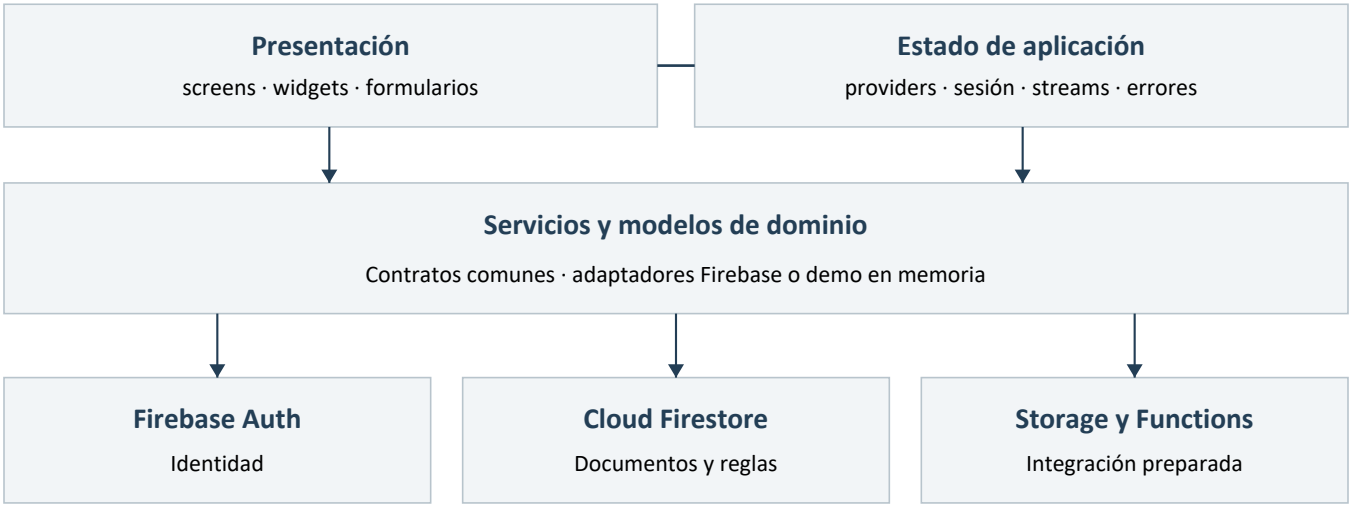


Figura 1 Dependencias de la aplicación. Storage y Cloud Functions tienen código preparado, pero su activación productiva permanece pendiente.

Responsabilidades del código

models contiene las entidades y la reconstrucción desde documentos. services define contratos y concentra consultas, escrituras y transacciones. providers expone estados de carga, error y sesión a través de Provider. screens y widgets presentan esos estados y capturan acciones del usuario. main.dart compone las dependencias y decide qué adaptadores se utilizan.

Los servicios Firebase y los servicios demo implementan contratos comunes, por ejemplo FeedService. Esto permite comprobar formularios e interacciones con datos en memoria y reservar los emuladores para validar reglas y persistencia. Los datos demo se pierden al recargar y no representan documentos existentes en el proyecto real.

Ciclo de vida del estado

Los providers cancelan suscripciones al cambiar de usuario o destruirse. Los flujos sensibles utilizan controles de generación para descartar respuestas de una sesión anterior. La puerta de acceso de la aplicación retira las pantallas privadas cuando finaliza la sesión. Así se evita que una respuesta tardía muestre publicaciones o puntos de otra cuenta.

5 Modelo de identidad y contenido

Firestore se organiza en documentos y subcolecciones. El UID de Firebase Auth vincula la identidad con la información del dominio, pero la contraseña y las credenciales del proveedor no se guardan en los documentos de la aplicación. Las fechas de nuevas acciones sociales se generan en el servidor para no depender del reloj del dispositivo.

Ruta lógica	Datos principales	Relación y visibilidad
users/{uid}	Nombre, email, photoUrl, bio y createdAt.	Perfil privado. Lectura y escritura únicamente del titular.
users/{uid}/pets/{petId}	ownerId, nombre, tipo, raza, edad, peso, photos, bio y fecha.	Uno a muchos desde usuario. Colección privada del propietario.
publicProfiles/{uid}	Solo name y photoUrl.	Proyección mínima del perfil, visible a miembros autenticados. Excluye email.
posts/{postId}	ownerId, autor, petId, petName, caption, imageUrl, mediaType, createdAt y likes.	Documento social visible a usuarios autenticados. imageUrl también referencia videos.
posts/{postId}/comments/{commentId}	userId, nombre y foto del autor, text y createdAt.	Uno a muchos desde publicación. Solo el autor puede eliminar su comentario.

Referencias e instantáneas

Cada publicación conserva tanto el identificador de la mascota como una instantánea de su nombre y de los datos visibles del autor. La desnormalización reduce lecturas al construir el feed. Su contrapartida es que una modificación posterior del perfil no reescribe automáticamente los nombres y fotografías históricos ya incorporados a publicaciones o comentarios.

Reacciones y orden cronológico

El contador de likes se calcula a partir de la lista de UID de cada publicación; no existe un entero independiente que el cliente pueda aumentar arbitrariamente. arrayUnion y arrayRemove evitan sobrescribir las reacciones de otros usuarios. El feed consulta createdAt en orden descendente y los comentarios se presentan por orden de creación mediante escuchas de snapshots.

Este esquema resulta adecuado para el volumen académico. Una lista de likes dentro de cada publicación y una consulta del feed sin paginación requieren revisión si aumenta la comunidad: una subcolección de reacciones y consultas paginadas permitirían reducir contención y lecturas.

6 Modelo de comunidad y consistencia

Ruta lógica	Contenido y regla de integridad
adoptions/{adoptionId}	Instantánea de la mascota, petId, ownerName, ubicación, requisitos y active. El propietario publica sobre una mascota propia y puede cerrar el aviso.
chats/{chatId}	Dos participants, fechas y resumen lastText, lastSenderId y lastMessageId. Los participantes no pueden reemplazarse después de la creación.
chats/{chatId}/messages/{id}	senderId, text y timestamp. Solo los miembros pueden leer y crear mensajes; no se habilita edición o borrado desde el cliente.
users/{uid}/activity/{kind}_{postId}	kind, postId, referenceId y createdAt. Crédito privado, único e inmutable. El puntaje se deriva de la actividad.
users/{uid}/devices/{id}	Token FCM, plataforma web y updatedAt. Datos privados del titular.
users/{uid}/notifications/{id}	Historial privado de avisos y estado de entrega. Escritura reservada al servidor mediante Admin SDK.
Storage users/{uid}/media/{archivo}	Archivos de imagen o video. Escritura por propietario y lectura autenticada según reglas; las URL de descarga con token pueden compartirse. Servicio productivo desactivado.

Adopciones y contacto

Los filtros combinan tipo, grupo etario y ubicación normalizada sin distinguir acentos. La clasificación de edad distingue menores de dos años, de dos a siete y de ocho en adelante. El formulario de interés entrega el mensaje por el chat privado con el responsable del aviso; no realiza un envío de correo electrónico.

Atomicidad de los mensajes

El identificador de conversación se obtiene de los dos UID ordenados y codificados, de manera que la misma pareja reutiliza su historial. Al enviar, el servicio escribe el documento del mensaje y el resumen del chat en un mismo lote. Las reglas consultan el estado posterior con getAfter para comprobar que el resumen corresponde al mensaje recién creado y a su remitente real.

La unidad de confirmación es el lote completo: si una validación falla, no debe quedar un resumen que anuncie un mensaje inexistente. Los textos se limitan a 2000 caracteres. Las pruebas cubren la sustitución de participantes, el acceso de una tercera cuenta y los intentos de modificar el historial.

7 Seguridad y privacidad

La autorización se ejecuta en Firestore y Storage, además de las comprobaciones de interfaz. Las reglas comparan request.auth.uid con el propietario o con los participantes y restringen los campos de los documentos sociales. Las fechas de creación de acciones se contrastan con request.time.

Amenaza	Control implementado	Evidencia o límite
Lectura de perfiles ajenos	Documento privado por UID y directorio separado con nombre y fotografía.	Pruebas de reglas con usuarios distintos. El feed se limita a miembros autenticados.
Suplantación de autor	UID del emisor vinculado a la sesión y mascota propia al publicar.	Se rechazan publicaciones sobre mascotas ajenas y comentarios con otro UID.
Manipulación de reacciones	Cambios limitados a agregar o quitar el UID propio.	Concurrencia, idempotencia y preservación de likes ajenos comprobadas.
Acceso a conversaciones	Lectura por participantes y validación atómica de mensaje y resumen.	Se rechaza el acceso de terceros y el cambio de participantes.
Inflación del puntaje	Créditos inmutables y fuente verificada; no se admite un total editable.	Se rechazan créditos duplicados, importes externos y referencias inválidas.
Archivos no permitidos	Propiedad, tipo MIME y límites de 10 MB para imagen y 50 MB para video.	Pruebas en Storage Emulator. El servicio productivo está desactivado.

Alcance de las garantías

El cliente no incluye claves de cuenta de servicio. La configuración web de Firebase y la clave pública VAPID identifican el proyecto y permiten inicializar SDK; no reemplazan las reglas de autorización. Los secretos administrativos corresponden al entorno servidor. El informe no reproduce tokens de dispositivos ni credenciales de usuarios.

Los perfiles y las mascotas privadas tienen autorización por propietario, pero no todas sus propiedades poseen validación exhaustiva de esquema en reglas. Los nombres y fotos históricos son instantáneas aportadas por el cliente. Las URL de descarga con token pueden compartirse y no deben tratarse como una garantía absoluta de confidencialidad del archivo.

Estas pruebas validan controles concretos del repositorio. No constituyen una prueba de penetración, una certificación de seguridad ni una comprobación de que las reglas remotas coincidan con la última versión local.

8 Gamificación y prevención de duplicados

La gamificación registra hechos y calcula el resultado a partir de ellos. Cada crédito se almacena bajo un identificador determinista compuesto por el tipo de acción y la publicación. El usuario no puede editar o borrar un crédito ya confirmado, ni enviar un puntaje total en su perfil.

Acción	Puntos	Condición para obtener el crédito
Publicar	10	Una publicación nueva del titular con fecha del servidor. El lote crea publicación y crédito.
Dar like	2	Primera acreditación sobre una publicación ajena. No se premian las reacciones sobre contenido propio.
Comentar	3	Primer crédito por comentar una publicación ajena. El comentario debe existir en la escritura validada.

El puntaje es la suma de créditos únicos: P = 10 × publicaciones + 2 × likes + 3 × comentarios acreditados. No depende de la cantidad actual de interacciones visibles.

Niveles e insignias

Elemento	Umbral o condición
Explorador · Amigo	Explorador desde 0 puntos; Amigo desde 25 puntos.
Embajador · Leyenda	Embajador desde 100 puntos; Leyenda desde 250 puntos.
Primera huella	Publicar por primera vez.
Buen compañero	Dar like a cinco publicaciones ajenas distintas.
Conversador	Comentar cinco publicaciones ajenas distintas.
Estrella PetVerse	Alcanzar 100 puntos.

Comportamiento ante acciones repetidas

Quitar un like o eliminar un comentario no retira un crédito obtenido. Repetir la interacción tampoco genera otro crédito para la misma publicación. La política evita sumar y restar un contador con cada toque y mantiene el logro como reconocimiento histórico. Las transacciones serializan la comprobación de créditos en interacciones concurrentes.

La incorporación de esta función no ejecuta una migración masiva de actividad anterior. Un like válido ya existente puede acreditarse una vez al interactuar nuevamente, porque la regla comprueba la existencia de la reacción y la ausencia del crédito, no necesariamente la fecha original del like. Las reglas y pruebas expresan este límite.

9 PWA y diseño adaptable

Instalación e identidad de la aplicación

El manifest declara nombre, idioma es-AR, identificador, ruta de inicio, ámbito y modo standalone. Se incluyen iconos PNG de 192 y 512 píxeles, variantes maskable y un icono de 180 píxeles para Apple Touch. Estos recursos pertenecen a la instalación web y no implican incorporar Apple como proveedor de autenticación.

El script pwa.js conserva el evento beforeinstallprompt y solicita la instalación desde el gesto del usuario. Distingue aceptación, cancelación y el evento appinstalled. Si el navegador no expone ese mecanismo, la interfaz muestra instrucciones manuales en lugar de anunciar una instalación inexistente. La integración Dart usa un adaptador web y una implementación alternativa para pruebas.

Caché y funcionamiento sin conexión

app-sw.js administra una lista explícita de recursos públicos como manifest, iconos y la página offline. La navegación prioriza la red y, si falla, presenta esa página. No almacena en su caché respuestas de Auth, Firestore, chats o archivos privados, ni ofrece editar publicaciones o enviar mensajes sin conexión. Tampoco se afirma que todos los mecanismos de persistencia de los SDK queden desactivados por esta decisión.

El bootstrap personalizado evita registrar el worker legado de Flutter sobre el mismo ámbito. El worker de Firebase Messaging mantiene un ámbito diferente. La preparación del build incorpora una versión derivada del contenido para invalidar la caché propia; la actualización espera que se cierren las pestañas anteriores para no mezclar recursos entre versiones.

Adaptación de la interfaz

Condición de ancho	Decisión de presentación
Menos de 900 px	Navegación inferior. Por debajo de 400 px se compactan etiquetas y espacios.
Desde 900 px	Barra lateral de navegación para aprovechar el ancho disponible.
Desde 1280 px	Barra lateral extendida; formularios y contenido mantienen límites legibles.
Formularios y grillas	Campos apilables, desplazamiento, textos con ajuste y columnas según el espacio disponible.

Las pruebas de widgets recorren perfil, feed, formularios, adopciones y chat en 320 × 740, 768 × 1024 y 1440 × 900. Esta evidencia cubre esos tamaños concretos; la instalación en dispositivos reales y una evaluación completa de accesibilidad siguen siendo verificaciones distintas.

10 Despliegue y operación

La aplicación está disponible en https://petverse-29db2.web.app/. Firebase Hosting sirve el frontend compilado y reescribe las rutas de la aplicación hacia index.html. La revisión del 20 de octubre de 2026 confirmó las siguientes respuestas públicas, sin iniciar sesión ni escribir datos reales.

Recurso comprobado	Resultado
Página principal por HTTPS	HTTP 200
manifest.json	HTTP 200
app-sw.js	HTTP 200
main.dart.js	HTTP 200
build-info.json	HTTP 200 · versión 930deeb24561994c8410

El build informa fecha 20 de octubre de 2026 a las 15:46:22 UTC. Una respuesta HTTP correcta acredita disponibilidad del recurso, no el éxito de todos los flujos autenticados.

Proceso reproducible de publicación

scripts/publish.ps1 prepara una copia temporal nueva fuera de OneDrive, excluye cachés y dependencias generadas y resuelve las herramientas antes de cambiar de directorio. Comprueba que la configuración del cliente corresponda al proyecto de destino. La compilación release fija una salida absoluta con --output para evitar depender de una configuración global de Flutter.

Antes de publicar, scripts/prepare_web.mjs exige main.dart.js, valida la estructura de iconos del manifest y compara los recursos del build con las fuentes actuales. Después incorpora el hash de versión al worker y genera build-info.json. Si una comprobación falla, el proceso se detiene y no anuncia un despliegue correcto.

La opción -Deploy ejecuta únicamente el despliegue de Hosting. No publica reglas ni funciones, no activa Storage y no habilita facturación. Los encabezados configurados incluyen no-cache para recursos que deben revalidarse, X-Content-Type-Options y Referrer-Policy. La disponibilidad del frontend no demuestra que el backend tenga activados todos los módulos.

Configuración y mantenimiento

La configuración local del cliente se conserva fuera del informe. Antes de presentar o actualizar la aplicación deben comprobarse los dominios autorizados de Authentication, la versión remota de las reglas y el acceso entre dos cuentas de prueba. Las copias de compilación son descartables; la fuente del proyecto sigue siendo la carpeta de trabajo y su historial de Git.

11 Estrategia de pruebas y resultados

La revisión del 20 de octubre de 2026 volvió a ejecutar 45 pruebas y todas finalizaron aprobadas. Se utilizaron datos ficticios o emuladores; no se enviaron mensajes a usuarios reales ni se modificó la base productiva. La copia de verificación de Flutter evita los bloqueos de archivos de la carpeta sincronizada.

Grupo	Casos	Resultado y cobertura
Flutter	24	Aprobadas. Autenticación demo, feed, comunidad, gamificación, instalación y pantallas adaptables.
Reglas de Firestore y Storage	9	Aprobadas. Propiedad, privacidad, concurrencia, créditos, chat, límites y progreso de subida emulada.
Ayudantes de notificaciones	4	Aprobadas. Eventos, identificadores estables, avisos y depuración de tokens permanentemente inválidos.
PWA en JavaScript	3	Aprobadas. Manifest e iconos, prompt de instalación y exclusión de rutas privadas del worker.
Preparación del build	5	Aprobadas. Salida explícita, build ausente, manifest antiguo, assets actuales y copia parcial rechazada.
Total	45	45 aprobadas y 0 fallidas.

Entorno y reproducibilidad

La implementación fue validada con Flutter 3.47.7 y Dart 3.13.5. Los ensayos de reglas utilizan Node.js 24 y Java 21, con el proyecto ficticio demo-petverse-tests y emuladores locales de Firestore y Storage. Cloud Functions declara Node.js 22 como entorno de ejecución; probar ayudantes con Node.js 24 no acredita por sí solo un despliegue o una ejecución completa en ese entorno servidor.

Los puntos de entrada reproducibles son flutter test, node --test test/web/*.test.cjs functions/notifications.test.cjs y npm test desde test/firestore. Los rechazos PERMISSION_DENIED producidos por los casos negativos de autorización son resultados esperados, no errores pendientes de la aplicación.

Qué no demuestran estas pruebas

La suite no sustituye una prueba integral con cuentas de producción, la recepción efectiva de push ni una subida al bucket real. Tampoco mide carga, latencia o accesibilidad conforme a una norma. El análisis estático y la compilación release tuvieron verificaciones previas satisfactorias; no se los contabiliza como casos adicionales dentro de las 45 pruebas.

12 Evidencia visual disponible

Las capturas conservadas corresponden a verificaciones locales y se identifican como tales. Son evidencia de PV11 y del fallback sin conexión de PV12. No acreditan datos de usuarios reales ni el funcionamiento del backend de notificaciones.

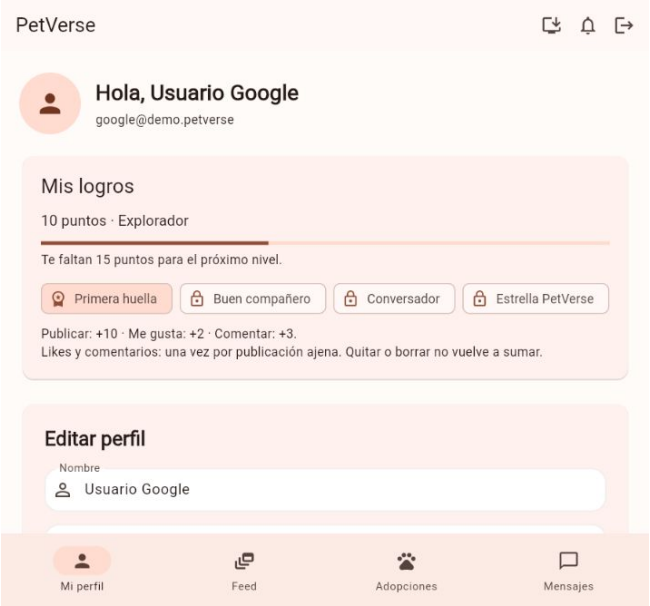


Figura 2 Gamificación en modo demo. La publicación acredita 10 puntos y desbloquea Primera huella. La tarjeta presenta el nivel y las condiciones de las insignias.



Figura 3 Verificación local del worker. Después de detener el servidor y recargar, se muestra la página offline explícita, sin reproducir conversaciones ni contenido privado.

El anexo visual debe ampliarse con acceso, perfil, mascotas, feed, adopciones y chat en el entorno de presentación. Para la aceptación final también se requiere la evidencia de instalación y de las integraciones pendientes.

13 Dificultades y riesgos pendientes

Dificultad observada	Causa técnica y solución aplicada
Bloqueos de compilación	OneDrive mantenía carpetas generadas en uso. La compilación se trasladó a una copia temporal, sin borrar la fuente ni depender de flutter clean para resolver el bloqueo.
Herramienta no disponible	La terminal no encontraba Flutter en PATH. Se utilizó el SDK instalado y se documentó cómo incorporarlo al entorno de la terminal.
Manifest antiguo y main.dart.js ausente	La salida efectiva de Flutter no coincidía con build/web por una configuración global. El script fija --output y verifica el archivo principal y la coherencia de los assets antes de desplegar.
Reacciones y créditos repetidos	Las escrituras completas podían perder cambios concurrentes. Transformaciones atómicas, transacciones e identificadores únicos conservan los likes ajenos y evitan créditos repetidos.
Workers con el mismo ámbito	El worker legado de Flutter podía competir con el worker PWA. Se adoptó un bootstrap propio y se separó el ámbito de Firebase Messaging.
Errores en pantallas estrechas	Los controles y textos rígidos desbordaban en móvil. Se aplicaron restricciones flexibles y pruebas de widgets en tres tamaños.

Restricciones operativas y deuda técnica

Storage y el envío automático de push requieren una decisión de activación independiente. El cliente FCM registra tokens con consentimiento y las funciones preparadas generan avisos para likes, comentarios y mensajes, pero ninguna prueba local permite afirmar que esos avisos lleguen al navegador en producción.

El backend preparado mantiene un historial de avisos con identificadores estables y elimina tokens definitivamente inválidos. Existe una ventana de reintento si el proceso se interrumpe después del envío y antes de registrar la entrega; una etiqueta estable reduce duplicados visibles, pero no garantiza entrega exactamente una vez.

Para una comunidad de mayor volumen deben evaluarse paginación del feed, consultas e índices, moderación de contenido, cuotas y monitoreo de costos. La política actual de puntos y la permanencia de los créditos también requieren mantenerse explícitas si se incorporan nuevas formas de interacción.

14 Conclusiones y referencias

PetVerse integra una interfaz Flutter modular con identidad administrada, documentos en tiempo real y reglas de autorización comprobadas en emuladores. La separación entre modelos, servicios, providers y pantallas permitió reutilizar los flujos en Firebase y en demostración sin mezclar datos temporales con persistencia real.

Los principales aprendizajes técnicos fueron la coordinación de escrituras atómicas, el diseño de controles por propietario y participante, y el manejo del ciclo de vida de los streams al cambiar de sesión. La gamificación se resolvió a partir de créditos verificables y no de un contador modificable. La PWA define de forma acotada su caché y su respuesta sin conexión, evitando prometer operaciones que requieren red.

El frontend publicado y las 45 pruebas aprobadas aportan una base verificable para la presentación. El cierre completo de la actividad sigue condicionado a activar y demostrar PV8 y PV10, instalar la PWA en un dispositivo real y completar el anexo visual y el video. La documentación de esas restricciones permite defender el alcance efectivo sin confundir código preparado con funcionalidad operativa.

Identificación de la entrega

Aplicación publicada: https://petverse-29db2.web.app/
Repositorio configurado: https://github.com/joacocop/PetVerse
Versión declarada del proyecto: 0.1.0+1
Corte técnico del informe: 20 de octubre de 2026.

Fuentes del proyecto

Consigna académica: Red Social para Mascotas con Flutter Web + Firebase, 14 de octubre de 2026, Prof. Jorge Fabián Siles Guzmán. Requisitos y entregables consultados en las páginas 15, 16, 19 y 20.

Implementación y verificaciones: lib/models, lib/services, lib/providers, lib/screens, firestore.rules, storage.rules, functions, web, scripts/publish.ps1, scripts/prepare_web.mjs y suites de test. Las guías PV7-PV10 y PV11-PV14 conservan las instrucciones operativas complementarias.

Documentación técnica complementaria

Arquitectura de Flutter

Firebase Authentication para Flutter

Reglas de Cloud Firestore

Transacciones y escrituras por lotes

Firebase Hosting

Firebase Cloud Messaging para Flutter